DTS manages the clocks in DCE. It consists of time servers that keep asking each other: "What time is it?" as well as other components. If DTS knows the maximum drift rate of each clock (which it does because it measures the drift rate), it can run the time calculations often enough to achieve the desired synchronization. For example, with clocks that are accurate to one part in a million, if no clock is to ever to be off by more than 10 msec, resynchronization must take place at least every three hours.

Actually, DTS must deal with two separate issues:

1. Keeping the clocks mutually consistent.
2. Keeping the clocks in touch with reality.

The first point has to do with making sure that all clocks return the same value when queried about the time (corrected for time zones, of course). The second has to do with ensuring that even if all the clocks return the same time, this time agrees with clocks in the real world. Having all the clocks agree that it is 12:04:00.000 is little consolation if it is actually 12:05:30.000. How DTS achieves these goals will be described below, but first we will explain what the DTS time model is and how programs interface to it.

10.4.1. DTS Time Model

Unlike most systems, in which the current time is simply a binary number, in DTS all times are recorded as intervals. When asked what the time is, instead of saying that it is 9:52, DTS might say it is somewhere between 9:51 and 9:53 (grossly exaggerated). Using intervals instead of values allows DTS to provide the user with a precise specification of how far off the clock might be.

Internally, DTS keeps track of time as a 64-bit binary number starting at the beginning of time. Unlike UNIX, in which time begins at 0000 on January 1, 1970, or TAI, which starts at 0000 on January 1, 1958, the beginning of time in DTS is 0000 on October 15, 1582, the date that the Gregorian calendar was introduced in Italy. (You never know when an old FORTRAN program from the 17th Century might turn up.)

People are not expected to deal with the binary representation of time. It is just used for storing times and comparing them. When displayed, times are shown in the format of Fig. 10-15. This representation is based on International Standard 8601 which solves the problem of whether to write dates as month/day/year (as in the United States) or day/month/year (everywhere else) by doing neither. It uses a 24-hour clock and records seconds accurately to 0.001 sec. It also effectively includes the time zone by giving the time difference from Greenwich Mean Time. Finally, and most important, the inaccuracy is given after the "I" in seconds. In this example, the inaccuracy is 5.000 seconds,

meaning that the true time might be anywhere from 3:29:55 P.M. to 3:30:05 P.M. In addition to absolute times, DTS also manages time differences, including the inaccuracy aspect.

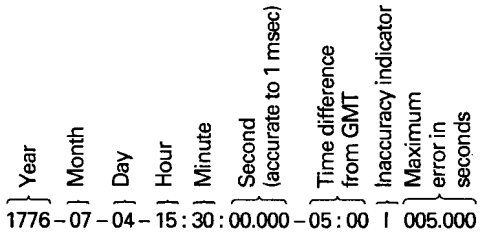

Fig. 10-15. Displaying time in DTS.

Recording times as intervals introduces a problem not present in other systems: it is not always possible to tell if one time is earlier than another. For example, consider the UNIX *make* program. Suppose that a source file has time interval 10:35:10 to 10:35:15 and the corresponding binary file has time interval 10:35:14 to 10:35:19. Is the binary file more recent? Probably, but not definitely. The only safe course of action for *make* is to recompile the source.

In general, when a program asks DTS to compare two times, there are three possible answers:

1. The first time is older.
2. The second time is older.
3. DTS cannot tell which is older.

Software using DTS has to be prepared to deal with all three possibilities. To provide backward compatibility with older software, DTS also supports a conventional interface with time represented as a single value, but using this value blindly may lead to errors.

DTS supports 33 calls (library procedures) relating to time. These calls are divided into the six groups listed in Fig. 10-16. We will now briefly mention these in turn. The first group gets the current time from DTS and returns it. The two procedures differ in how the time zone is handled. The second group handles time conversion between binary values, structured values, and ASCII values. The third group makes it possible to present two times as input and get back a time whose inaccuracy spans the full range of possible times. The fourth group compares two times, with or without using the inaccuracy part. The fifth group provides a way to add two times, subtract two times, multiply a relative time by a constant, and so on. The last group manages time zones.

Group	# Calls	Description
Retreiving times	2	Get the time
Converting times	18	Binary-ASCII conversion
Manipulating times	3	Interval arithmetic
Comparing times	2	Compare two times
Calculating times	5	Arithmetic operations on times
Using time zones	3	Time zone management

Fig. 10-16. Groups of time-related calls in DTS.

10.4.2. DTS Implementation

The DTS service consists (conceptually) of several components. The **time clerk** is a daemon process that runs on client machines and keeps the local clock synchronized with the remote clocks. The clerk also keeps track of the linearly growing uncertainty of the local clock. For example, a clock with a relative error of one part in a million might gain or lose as much as 3.6 msec per hour, as discussed above. When the time clerk calculates that the possible error has passed the bound of what is allowed, it resynchronizes.

A time clerk resynchronizes by contacting all the **time servers** on its LAN. These are daemons whose job it is to keep the time consistent and accurate within known bounds. For example, in Fig. 10-17 a time clerk has asked for and received the time from four time servers. Each one provides an interval in which it believes that UTC falls. The clerk computes its new value of time as follows. First, values that do not overlap with any over values (such as source 4) are discarded as being untrustworthy. Then the largest intersection falling within the remaining intervals is computed. The clerk then sets its value of UTC to the midpoint of this interval.

After resynchronizing, a clerk has a new UTC that is usually either ahead or behind its current one. It could just set the clock to the new value, but generally doing so is unwise. First of all, this might require setting the clock backward, which means that files created after the resynchronization might appear to be older than files created just before it. Programs such as *make* will behave incorrectly under these circumstances.

Even if the clock has to be set forward, it is better not to do it abruptly because some programs display information and then give the user a certain number of seconds to react. Having this interval sharply reduced could, for

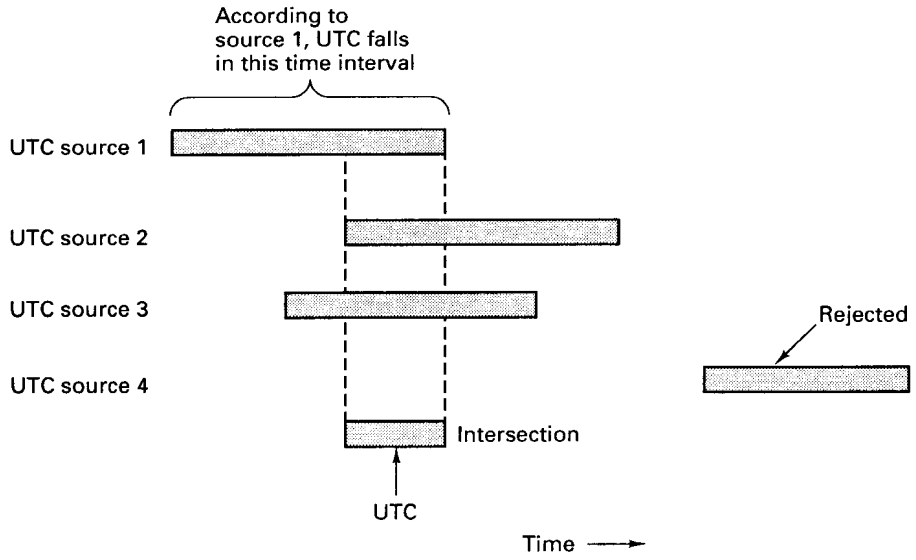

Fig. 10-17. Computation of the new UTC from four time sources.

example, cause an automated test system to display a question on the screen for a student, and then immediately time out, telling the student that he had taken too long to answer it.

Consequently, DTS can make the correction gradually. For example, if the clock is 5 sec behind, instead of adding 10 msec to it 100 times a second, 11 msec could be added at each tick for the next 50 seconds.

Time servers come in two varieties, local and global. The local ones participate in timekeeping within their cells. The global ones keep local time servers in different cells synchronized. The time servers communicate among themselves periodically to keep their clocks consistent. They also use the algorithm of Fig. 10-17 for choosing the new UTC.

Although it is not required, best results are obtained if one or more global servers are directly connected to a UTC source via a satellite, radio, or telephone connection. DTS defines a special interface, the **time provider interface**, which defines how DTS acquires and distributes UTC from external sources.

10.5. DIRECTORY SERVICE

A major goal of DCE is to make all resources accessible to any process in the system, without regard to the relative location of the resource user (client) and the resource provider (server). These resources include users, machines,

cells, servers, services, files, security data, and many others. To accomplish this goal, it is necessary for DCE to maintain a directory service that keeps track of where all resources are located and to provide people-friendly names for them. In this section we will describe this service and how it operates.

The DCE directory service is organized per cell. Each cell has a **Cell Directory Service** (**CDS**), which stores the names and properties of the cell's resources. This service is organized as a replicated, distributed data base system to provide good performance and high availability, even in the face of server crashes. To operate, every cell must have at least one running CDS server.

Each resource has a unique name, consisting of the name of its cell followed by the name used within its cell. To locate a resource, the directory service needs a way to locate cells. Two such mechanisms are supported, the **Global Directory Service** (**GDS**) and the **Domain Name System** (**DNS**). GDS is the "native" DCE service for locating cells. It uses the X.500 standard. However, since many DCE users use the Internet, the standard Internet naming system, DNS, is also supported. It would have been better to have had a single mechanism for locating cells (and a single syntax for naming them), but political considerations made this impossible.

The relationship between these components is illustrated in Fig. 10-18. Here we see another component of the directory service, the **Global Directory Agent** (**GDA**), which CDS uses to interact with GDS and DNS. When CDS needs to look up a remote name, it asks its GDA to do the work for it. This design makes CDS independent of the protocols used by GDS and DNS. Like CDS and GDS, GDA is implemented as a daemon process that accept queries using RPC and returns replies.

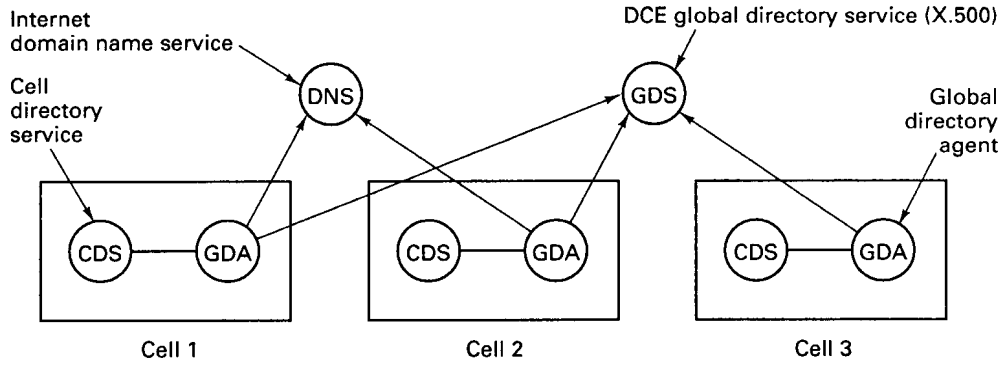

Fig. 10-18. Relation between CDS, GDS, GDA, and DNS.

In the following section we will describe how names are formed in DCE. After that, we will examine CDS and GDS.

10.5.1. Names

Every resource in DCE has a unique name. The set of all names forms the DCE namespace. Each name can have up to five parts, some of which are optional. The five parts are shown in Fig. 10-19.

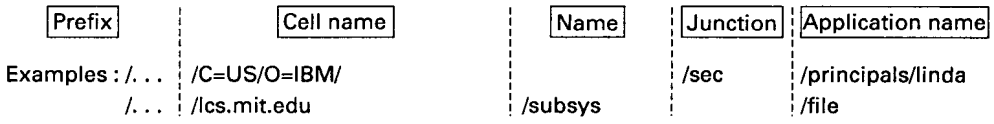

Fig. 10-19. DCE names can have up to five parts.

The first part is the **prefix**, which tells whether the name is global to the entire DCE namespace or local to the current cell. The prefix /... indicates a global name, whereas the prefix /.: denotes a local name. A global name must contain the name of the cell needed; a local name must not. When a request comes in to CDS, it can tell from the prefix whether it can handle the request itself or whether it must pass it to the GDA for remote lookup by GDS.

Cell names can be specified either in X.500 notation or in DNS notation. Both systems are highly elaborate, but for our purposes the following brief introduction will suffice. X.500 is an international standard for naming. It was developed within the world of telephone companies to provide future telephone customers with an electronic phonebook. It can be used for naming people, computers, services, cells, or anything else needing a unique name.

Every named entity has a collection of attributes that describe it. These can include its country (e.g., US, GB, DE), its organization (e.g., IBM, HARVARD, DOD), its department (e.g., CS, SALES, TAX), as well as more detailed items such as employee number, supervisor, office number, telephone number, and name. Each attribute has a value. An X.500 name is a list of *attribute=value* items separated by slashes. For example,

/C=US/O=YALE/OU=CS/TITLE=PROF/TELEPHONE=3141/OFFICE=210/SURNAME=LIN/

might describe Professor Lin in the Yale Computer Science Department. The attributes C, O, and OU are present in most names and refer to country, organization, and organization unit (department), respectively.

The idea behind X.500 is that a query must supply enough attributes that the target is uniquely specified. In the example above, C, O, OU, and SURNAME might do the job, but C, O, OU, and OFFICE might work, too, if the requester had forgotten the name but remembered the office number. Providing all the attributes except the country and expecting the server to search the entire world for a match is not sporting.

DNS is the Internet's scheme for naming hosts and other resources. It

divides the world up into top-level domains consisting of countries and in the United States, EDU (educational institutions), COM (companies), GOV (government), MIL (military sites), plus a few others. These, in turn, have subdomains such as *harvard.edu*, *princeton.edu*, and *stanford.edu*, and subsubdomains such *cs.cmu.edu*. Both X.500 and DNS can be used to specify cell names. In Fig. 10-19 the two example cells might be the tax department at IBM and the Laboratory for Computer Science at M.I.T.

The next level of name is usually the name of a standard resource or a junction, which is analogous to a mount point in UNIX, causing the search to switch over to a different naming system, such as the file system or the security system. Finally, comes the resource name itself.

10.5.2. The Cell Directory Service

The CDS manages the names for one cell. These are arranged as a hierarchy, although as in UNIX, symbolic links (called **soft links**) also exist. An example of the top of the tree for a simple cell is shown in Fig. 10-20.

The top level directory contains two profile files containing RPC binding information, one of them topology independent and one of them reflecting the network topology for applications where it is important to select a server on the client's LAN. It also contains an entry telling where the CDS data base is. The *hosts* directory lists all the machines in the cell, and each subdirectory there has an entry for the host's RPC daemon (*self*) and default profile (*profile*), as well as various parts of the CDS system and the other machines. The junctions provide the connections to the file system and security data base, as mentioned above, and the *subsys* directory contains all the user applications plus DCE's own administrative information.

The most primitive unit in the directory system is the **CDS directory entry**, which consists of a name and a set of attributes. The entry for a service contains the name of the service, the interface supported, and the location of the server.

It is important to realize that CDS only holds information about resources. It does not provide access to the resources themselves. For example, a CDS entry for *printer23* might say that it is a 20-page/min, 600-dot/inch color laser printer located on the second floor of Toad Hall with network address 192.30.14.52. This information can be used by the RPC system for binding, but to actually use the printer, the client must do an RPC with it.

Associated with each entry is a list of who may access the entry and in what way (e.g., who may delete the entry from the CDS directory). This protection information is managed by CDS itself. Getting access to the CDS entry does not ensure that the client may access the resource itself. It is up to the server that manages the resource to decide who may use the resource and how.

A group of related entries can be collected together into a **CDS directory**.

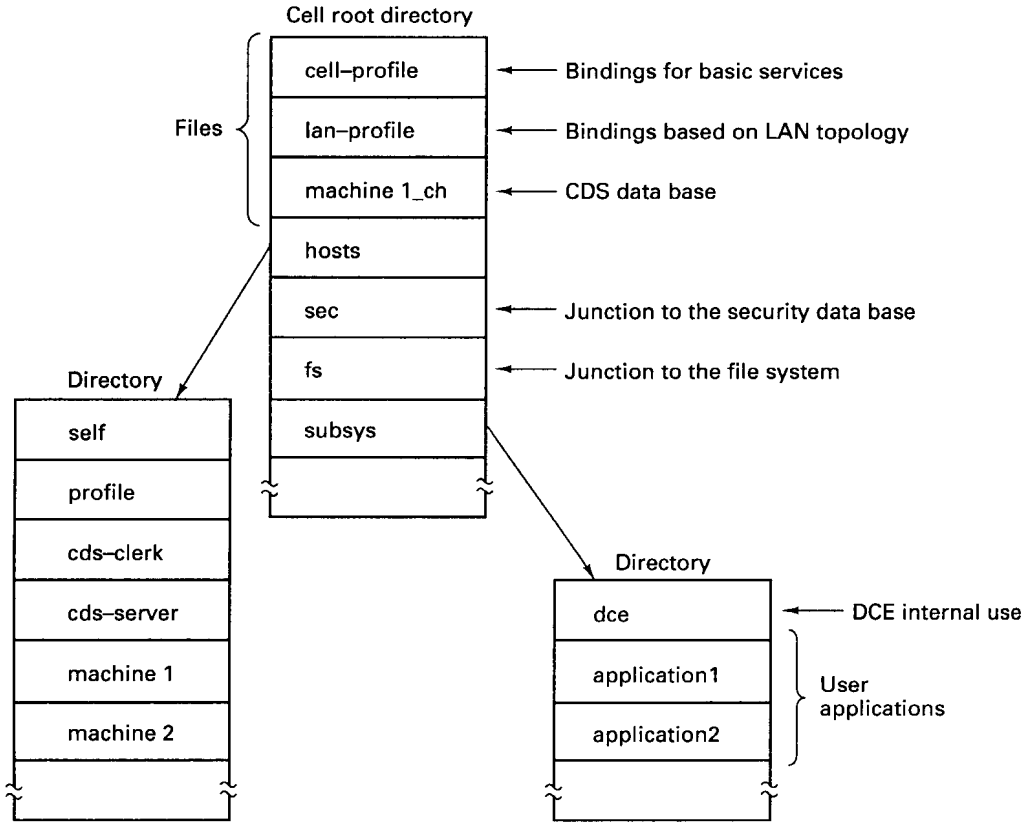

Fig. 10-20. The namespace of a simple DCE cell.

For example, all the printers might be grouped for convenience into a directory *printers*, with each entry describing a different printer or group of printers.

CDS permits entries to be replicated to provide higher availability and better fault tolerance. The directory is the unit of replication, with an entire directory either being replicated or not. For this reason, directories are a heavier weight concept than in say, UNIX. CDS directories cannot be created and deleted from the usual programmer's interface. Special administrative programs are used.

A collection of directories forms a **clearinghouse**, which is a physical data base. A cell may have multiple clearinghouses. When a directory is replicated, it appears in two or more clearinghouses.

The CDS for a cell may be spread over many servers, but the system has been designed so that a search for any name can begin by any server. From the prefix it is possible to see if the name is local or global. If it is global, the request is passed on to the GDA for further processing. If it is local, the root

directory of the cell is searched for the first component. For this reason, every CDS server has a copy of the root directory. The directories pointed to by the root can either be local to that server or on a different server, but in any event, it is always possible to continue the search and locate the name.

With multiple copies of directories within a cell, a problem occurs: How are updates done without causing inconsistency? DCE takes the easy way out here. One copy of each directory is designated as the master; the rest are slaves. Both read and update operations may be done on the master, but only reads may be done on the slaves. When the master is updated, it tells the slaves.

Two options are provided for this propagation. For data that must be kept consistent all the time, the changes are sent out to all slaves immediately. For less critical data, the slaves are updated later. This scheme, called **skulking**, allows many updates to be sent together in larger and more efficient messages.

CDS is implemented primarily by two major components. The first one, the **CDS server**, is a daemon process that runs on a server machine. It accepts queries, looks them up in its local clearinghouse, and sends back replies.

The second one, the **CDS clerk**, is a daemon process that runs on the client machine. Its primary function is to do client caching. Client requests to look up data in the directory go through the clerk, which then caches the results for future use. Clerks learn about the existence of CDS servers because the latter broadcast their location periodically.

As a simple example of the interaction between client, clerk, and server, consider the situation of Fig. 10-21(a). To look up a name, the client does an RPC with its local CDS clerk. The clerk then looks in its cache. If it finds the answer, it responds immediately. If not, as shown in the figure, it does an RPC over the network to the CDS server. In this case the server finds the requested name in its clearinghouse and sends it back to the clerk, which caches it for future use before returning it to the client.

In Fig. 10-21(b), there are two CDS servers in the cell, but the clerk only knows about one of them—the wrong one, as it turns out. When the CDS server sees that it does not have the directory entry needed, it looks in the root directory (remember that all CDS servers have the full root directory), to find the soft link to the correct CDS server. Armed with this information, the clerk tries again (message 4). As before, the clerk caches the results before replying.

10.5.3. The Global Directory Service

In addition to CDS, DCE has a second directory service, GDS, which is used to locate remote cells, but can also be used to store other arbitrary directory information. GDS is important because it is the DCE implementation of X.500, and as such, can interwork with other (non-DCE) X.500 directory services. X.500 is defined in International Standard ISO 9594.

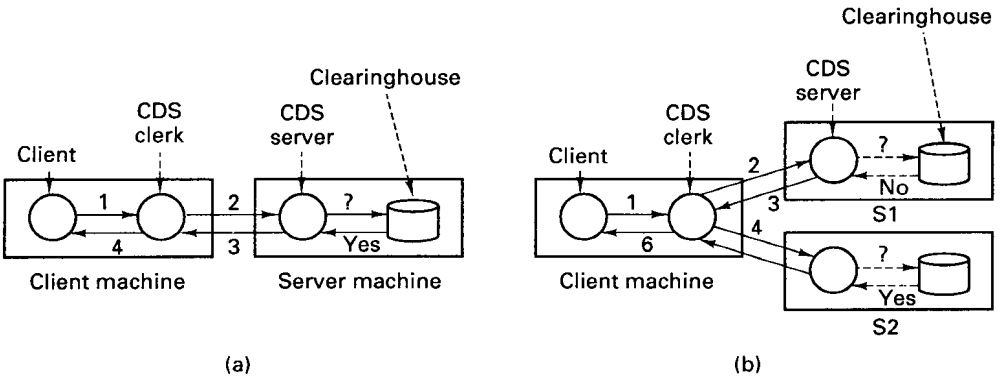

Fig. 10-21. Two examples of a client looking up a name. (a) The CDS server first contacted has the name. (b) The CDS server first contacted does not have the name.

X.500 uses an object-oriented information model. Every item stored in an X.500 directory is an object. An object can be a country, a company, a city, a person, a cell, or a server, for example.

Every object has one or more attributes. An attribute consists of a type and a value (or sometimes multiple values). In written form, the type and value are separated by an equal sign, as in *C=US* to indicate that the type is *country* and the value is *United States*.

Objects are grouped into classes, with all the objects of a given class referring to the same "kind" of object. A class can have mandatory attributes, such as a zipcode for a post office object, and optional attributes, such as a FAX number for a company object. The object class attribute is always mandatory.

The X.500 naming structure is hierarchical. A simple example is given in Fig. 10-22. Here we have shown just two of the entries below the root, a country (US) and an organization (IBM). The decision of which object to put where in the tree is not part of X.500, but is up to the relevant registration authority. For example, if a newly-formed company, say, Invisible Graphics, wishes to register to be just below *C=US* in the worldwide tree, it must contact ANSI to see if that name is already in use, and if not, claim it and pay a registration fee.

Paths through the naming tree are given by a sequence of attributes separated by slashes, as we saw earlier. In our current example, Joe of Joe's Deli in San Francisco would be

/C=US/STATE=CA/LOC=SF/O=JOE'S-DELI/CN=JOE/

where *LOC* indicates a location and *CN* is the (common) name of the object. In X.500 jargon, each component of the path is called the **RDN (Relative**

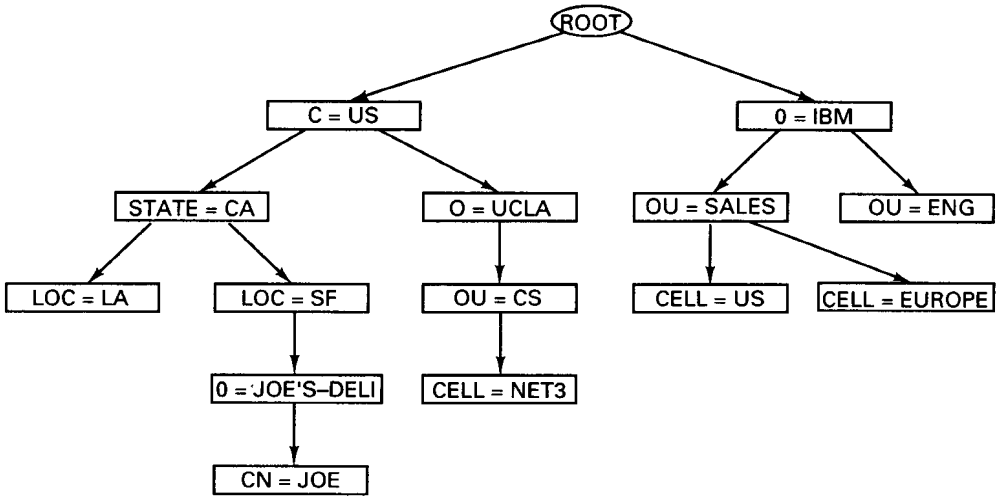

Fig. 10-22. An X.500 directory information tree.

Distinguished Name) and the full path is called the **DN** (**Distinguished Name**). All the RDNs originating at any given object must be distinct, but RDNs originating at different objects may be the same.

In addition to normal objects, the X.500 tree can contain **aliases**, which name other objects. Aliases are similar to symbolic links in a file system.

The structure and properties of an X.500 directory information tree are defined by its **schema**, which consists of three tables:

1. Structure Rule Table—Where each object belongs in the tree.
2. Object Class Table —Inheritance relationships among the classes.
3. Attribute Table —Specifies the size and type of each attribute.

The Structure Rule Table is basically a description of the tree in table form and primarily tells which object is a child of which other object. The Object Class Table describes the inheritance hierarchy of the objects. For example, there might conceivably be a class telephone number, with subclasses voice and FAX.

Note that the example of Fig. 10-22 contains no information at all about the object inheritance hierarchy since organizations there occur under country, location, and the root. The structure depicted in this figure would be reflected in the Structure Rule Table. The Object Class Table might be set up with organizational unit as a subclass of an organization, and cell as a subclass of organizational unit, but this information cannot be derived from the figure. In addition to

telling which class a given class is derived from, the Object Class Table lists its unique object identifier, and its mandatory and optional attributes.

The Attribute Table tells how many values each attribute may have, how much memory they may occupy, and what their types are (e.g., integers, Booleans, or reals). Attributes are described in the OSI ASN.1 notation. DCE provides a compiler from ASN.1 to C (MAVROS), which is analogous to its IDL compiler for RPC stubs. This compiler is really for use in building DCE; normally, users do not encounter it.

Each attribute can be marked as PUBLIC, STANDARD, or SENSITIVE. It is possible to associate with each object access control lists specifying which users may read and which users may modify attributes in each of these three categories.

The standard interface to X.500, called **XOM (X/Open Object Management)**, is supported. However, the usual way for programs to access the GDS system is via the **XDS (X/Open Directory Server)** library. When a call is made to one of the XDS procedures, it checks to see if the entry being manipulated is a CDS entry or a GDS entry. If it is CDS, it just does the work directly. If it is GDS, it makes the necessary XOM calls to get the job done.

The XDS interface is amazingly small, only 13 calls (versus 101 calls and a 409-page manual for DCE RPC). Of these, five set up and initialize the connection between the client and the directory server. The eight calls that actually use directory objects are listed in Fig. 10-23.

Call	Description
Add_entry	Add an object or alias to the directory
Remove_entry	Remove an object or alias from the directory
List	List all the objects directly below a specified object
Read	Read the attributes of a specified object
Modify_entry	Atomically change the attributes of a specified object
Compare	Compare a certain attribute value with a given one
Modify_rdn	Rename an object
Search	Search a portion of the tree for an object

Fig. 10-23. The XDS calls for manipulating directory objects.

The first two calls add and delete objects from the directory tree, respectively. Each call specifies a full path so there is never any ambiguity about which object is being added or deleted. The *list* call lists all the objects directly under the object specified. *Read* and *modify_entry* read and write the attributes of the specified object, copying them from the tree to the caller, or vice versa. *Compare* examines a particular attribute of a specified object, compares it to a given value, and tells whether the two match or not. *Modify_rdn* changes one relative distinguished name, for example, changing the path $a/b/c$ to $a/x/c$. Finally, *search* starts at a given object and searches the object tree below it (or a portion of it) looking for objects that meet a given criterion.

All of these calls operate by first determining whether CDS or GDS is needed. X.500 names are handled by GDS; DNS or mixed names are handled by CDS, as illustrated in Fig. 10-24. First let us trace the lookup of a name in X.500 format. The XDS library sees that it needs to look up an X.500 name, so it calls the **DUA (Directory User Agent)**, a library linked into the client code. This handles GDS caching, analogous to the CDS clerk, which handles CDS caching. Users have more control over GDS caching than they do over CDS caching and can, for example, specify which items are to be cached. They can even bypass the DUA if it is absolutely essential to get the latest data.

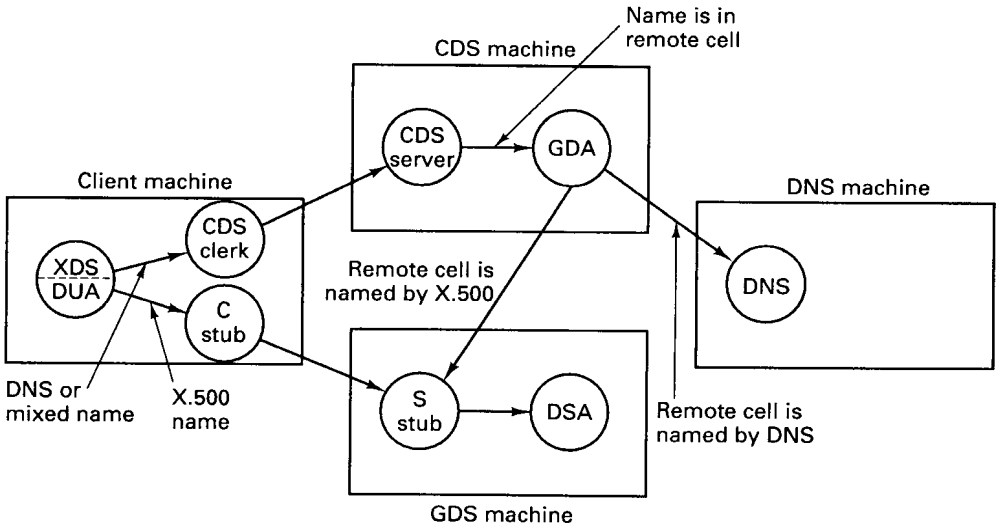

Fig. 10-24. How the servers involved in name lookup invoke one another.

Analogous to the CDS server, there is a **DSA (Directory Server Agent)** that handles incoming requests from DUAs, from both its own cell and from remote ones. If a request cannot be handled because the information is not

available, the DSA either forwards the request to the proper cell or tells the DUA to do so.

In addition to the DUA and DSA processes, there are also separate stub processes that handle the wide-area communication using the OSI ASCE and ROSE protocols on top of the OSI transport protocols.

Now let us trace the lookup of a DNS or mixed name. XDS does an RPC with the CDS clerk to see if it is cached. If it is not, the CDS server is asked. If the CDS server sees that the name is local, it looks it up. If, however, it belongs to a remote cell, it asks the GDA to work on it. The GDA examines the name of the remote cell to see whether it is specified as a DNS name or an X.500 name. In the former case it asks the DNS server to find a CDS server in the cell; in the latter case it uses the DSA. All in all, name lookup is a complex business.

10.6. SECURITY SERVICE

In most distributed systems, security is a major concern. The system administrator may have definite ideas about who can use which resource (e.g., no lowly undergraduates using the fancy color laser printer), and many users may want their files and mailboxes protected from prying eyes. These issues arise in traditional timesharing systems too, but there they are solved simply by having the kernel manage all the resources. In a distributed system consisting of potentially untrustworthy machines communicating over an insecure network, this solution does not work. Nevertheless, DCE provides excellent security. In this section we will examine how that is accomplished.

Let us begin our study by introducing a few important terms. In DCE, a **principal** is a user or process that needs to communicate securely. Human beings, DCE servers (such as CDS), and application servers (such as the software in a automated teller machine in a banking system) can all be principals. For convenience, principals with the same access rights can be collected together in groups. Each principal has a **UUID** (**Unique User IDentifier**), which is a binary number associated with it and no other principal.

Authentication is the process of determining if a principal really is who he/she/it claims to be. In a timesharing system, a user logs in by typing his login name and password. A simple check of the local password file tells whether the user is lying or not. After a user logs in successfully, the kernel keeps track of the user's identity and allows or refuses access to files and other resources based on it.

In DCE a different authentication procedure is necessary. When a user logs in, the login program verifies the user's identity using an authentication server. The protocol will be described later, but for the moment it is sufficient to say that it does *not* involve sending the password over the network. The DCE

authentication procedure uses the Kerberos system developed at M.I.T. (Kohl, 1991; and Steiner et al., 1988). Kerberos, in turn, is based on the ideas of Needham and Schroeder (1978). For other approaches to authentication, see (Lampson et al., 1992; Wobber et al., 1994; and Woo and Lam, 1992).

Once a user has been authenticated, the question of which resources that user may access, and how, comes up. This issue is called **authorization**. In DCE, authorization is handled by associating an **ACL (Access Control List)** with each resource. The ACL tells which users, groups, and organizations may access the resource and what they may do with it. Resources may be as coarse as files or as fine as data base entries.

Protection in DCE is closely tied to the cell structure. Each cell has one **security service** that the local principals have to trust. The security service, of which the authentication server is part, maintains keys, passwords and other security-related information in a secure data base called the **registry**. Since different cells can be owned by different companies, communicating securely from one cell to another requires a complex protocol, and can be done only if the two cells have set up a shared secret key in advance. For simplicity, we will restrict our subsequent discussion to the case of a single cell.

10.6.1. Security Model

In this section we will review briefly some of the basic principles of **cryptography**, the science of sending secret messages, and the DCE requirements and assumptions in this area. Let us assume that two parties, say a client and a server, wish to communicate securely over an insecure network. What this means is that even if an intruder (e.g., a wiretapper) manages to steal messages, he will not be able to understand them. Stronger yet, if the intruder tries to impersonate the client or if the intruder records legitimate messages from the client and plays them back for the server later, the server will be able to see this and reject these messages.

The traditional cryptographic model is shown in Fig. 10-25. The client has an unencrypted message, P, called the **plaintext**, which is transformed by an encryption algorithm parametrized by a key, K. The encryption can be done by the client, the operating system, or by special hardware. The resulting message, C, called the **ciphertext** is unintelligible to anyone not possessing the key. When the ciphertext arrives at the server, it is decrypted using K, which results in the original plaintext. The notation generally used to indicate encryption is

Ciphertext = {Plaintext} Key

that is, the string inside the curly brackets is the plaintext, and the key used is written afterward.

Cryptographic systems in which the client and server use different keys

(e.g., public key cryptography) also exist, but since they are not used in DCE, we will not discuss them further.

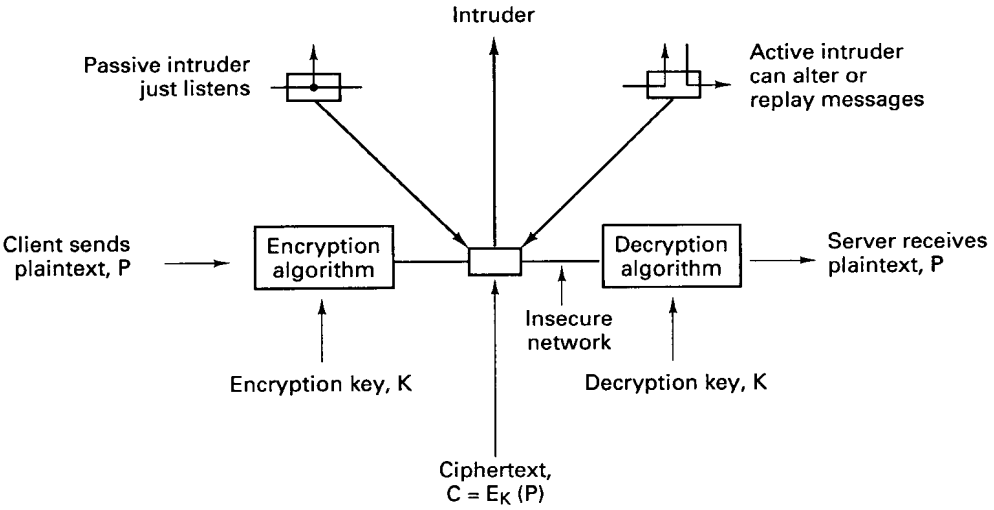

Fig. 10-25. A client sending an encrypted message to a server.

It is probably worthwhile to make some of the assumptions underlying this model explicit, to avoid any confusion. We assume that the network is totally insecure, and that a determined intruder can capture any message sent on it, possibly removing it as well. The intruder can also inject arbitrary new messages and replay old ones to his heart's content.

Although we assume that most of the major servers are moderately secure, we explicitly assume that the security server (including its disks) can be placed in a tightly locked room guarded by a nasty three-headed dog (Kerberos of Greek mythology) and that no intruder can tamper with it. Consequently, it is permitted for the security server to know each user's password, even though the passwords cannot be sent over the network. It is also assumed that users do not forget their passwords or accidentally leave them lying around the terminal room on bits of paper. Finally, we assume that clocks in the system are roughly synchronized, for example, using DTS.

As a consequence of this hostile environment, a number of requirements were placed on the design from its inception. The most important of these are as follows. First, at no time may user passwords appear in plaintext (i.e., unencrypted) on the network or be stored on normal servers. This requirement precludes doing authentication just by sending user passwords to an authentication server for approval.

Second, user passwords may not even be stored on client machines for more

than a few microseconds, for fear that they might be exposed in a core dump should the machine crash.

Third, authentication must work both ways. That is, not only must the server be convinced who the client is, but the client must also be convinced who the server is. This requirement is necessary to prevent an intruder from capturing messages from the client and pretending that it is, say, the file server.

Finally, the system must have firewalls built into it. If a key is somehow compromised (disclosed), the damage done must be limited. This requirement can be met, for example, by creating temporary keys for specific purposes and with short lifetimes, and using these for most work. If one of these keys is ever compromised, the potential for damage is restricted.

10.6.2. Security Components

The DCE security system consists of several servers and programs, the most important of which are shown in Fig. 10-26. The **registry server** manages the security data base, the registry, which contains the names of all the principals, groups, and organizations. For each principal, it gives account information, groups and organizations the principal belongs to, whether the principal is a client or a server, and other information. The registry also contains policy information per cell, including the length, format, and lifetime for passwords and related information. The registry can be thought of as the successor to the password file in UNIX (*/etc/passwd*). It can be edited by the system administrator using the registry editor. These can add and delete principals, change keys, and so on.

The **authentication server** is used when a user logs in or a server is booted. It verifies the claimed identity of the principal and issues a kind of ticket (described below) that allows the principal to do subsequent authentication without having to use the password again. The authentication server is also known as the **ticket granting server** when it is granting tickets rather than authenticating users, but these two functions reside in the same server.

The **privilege server** issues documents called **PACs (Privilege Attribute Certificates)** to authenticated users. PACs are encrypted messages that contain the principal's identity, group membership, and organizational membership in such a way that servers are instantly convinced without need for presenting any additional information. All three of these servers run on the security server machine in the locked room with the mutant dog outside.

The **login facility** is a program that asks users their names and passwords during the login sequence. It uses the authentication and privilege servers to do its job, which is to get the user logged in and to collect the necessary tickets and PACs for them.

Once a user is logged in, he can start a client process that can communicate

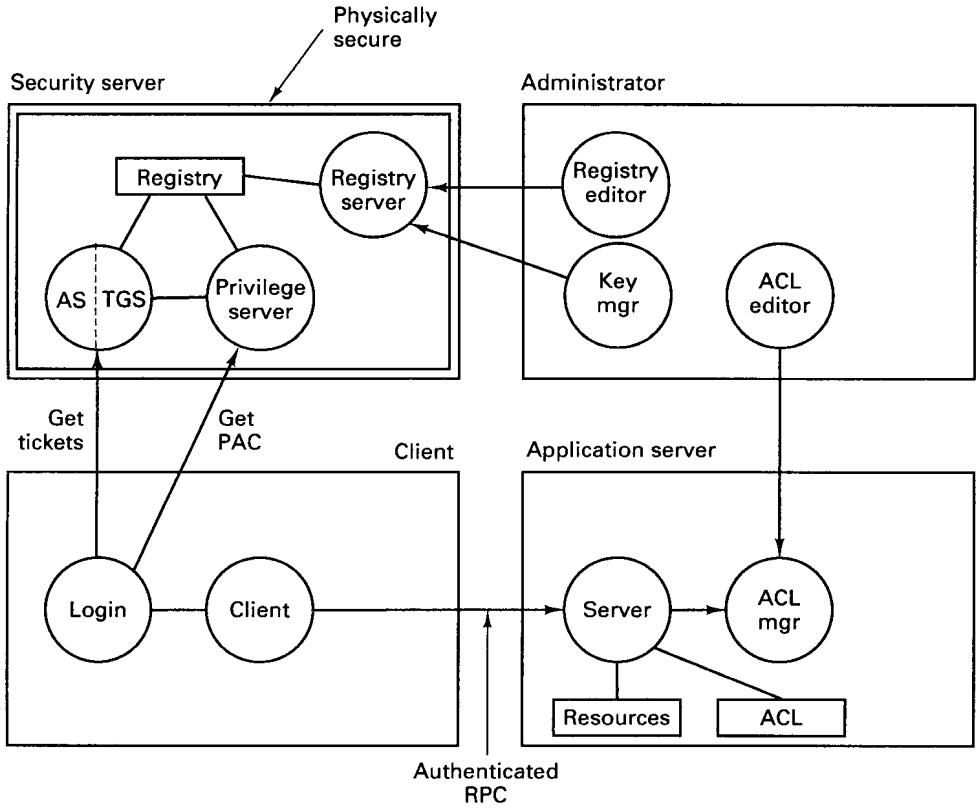

Fig. 10-26. Major components of the DCE security system for a single cell.

securely with a server process using authenticated RPC. When an authenticated RPC request comes in, the server uses the PAC to determine the user's identity, and then checks its ACL to see if the requested access is permitted. Each server has its own ACL manager for guarding its own objects. Users can be added or removed from an ACL, permissions granted or removed, and so on, using an ACL editor program.

10.6.3. Tickets and Authenticators

In this section we will describe how the authentication and privilege servers work and how they allow users to log into DCE in a secure way over an insecure network. The description covers only the barest essentials, and ignores most of the variants and options available.

Each user has a secret key known only to himself and to the registry. It is

computed by passing the user's password through a one-way (i.e., noninvertible) function. Servers also have secret keys. To enhance their security, these keys are used only briefly, when a user logs in or a server is booted. After that authentication is done with tickets and PACs.

A **ticket** is an encrypted data structure issued by the authentication server or ticket-granting server to prove to a specific server that the bearer is a client with a specific identity. Tickets have many options, but mostly we will discuss tickets that look like this:

ticket = S, {session-key, client, expiration-time, message-id}K_S

where S is the server for whom the ticket is intended. The information within curly braces is encrypted using the server's private key, K_S. The fields encrypted include a temporary session key, the client's identity, the time at which the ticket ceases to be valid, and a message identifier or **nonce** that is used to relate requests and replies. When the server decrypts the ticket with its private key, it obtains the session key to use when talking to the client. In our descriptions below, we will omit all encrypted ticket fields except the session-key and client name.

In some situations tickets and PACs are used together. Tickets establish the identity of the sender (as an ASCII string), whereas PACs give the numerical values of user-id and group-ids that a particular principal is associated with. Tickets are generated by the authentication server or ticket-granting server (one and the same server), whereas PACs are generated by the privilege server.

In many situations, it is not essential that messages be secret, only that intruders not be able to forge or modify them. For this purpose, authenticators can be attached to plaintext messages to prevent active intruders from changing them. An **authenticator** is an encrypted data structure containing at least the following information:

authenticator = {sender, MD5-checksum, timestamp} K

where the checksum algorithm, **MD5** (**Message Digest 5**), has the property that given a 128-bit MD5 checksum it is computationally infeasible to modify the message so that it still matches the checksum (which is protected by encryption). The timestamp is needed to make it possible for the receiver to detect the replay of an old authenticator.

10.6.4. Authenticated RPC

The sequence from logging in to the point where the first authenticated RPC is possible typically requires five steps. Each step consists of a message from the client to some server, followed by a reply back from the server to the client. The steps are summarized in Fig. 10-27 and are described below. For

simplicity, most of the fields, including the cells, message IDs, lifetimes, and identifiers have been omitted. The emphasis is on the security fundamentals: keys, tickets, authenticators, and PACs.

Principals
A: Authentication server (handles authentication)
C: Client (user)
P: Privilege server (issues PAC's)
S: Application server (does the real work)
T: Ticket-granting server (issues tickets)

Step 1: Client gets a ticket for the ticket-granting server using a password
C → A: C (just the client's name, in plaintext)
C ← A: $\{K_1, \{K_1, C\}K_A\}\ K_C$

Step 2: Client uses the previous ticket to get a ticket for the privilege server
C → T: $\{K_1, C\}K_A$, {C, MD-5 checksum, Timestamp}K_1
C ← T: $\{K_2,\{C, K_2\}K_p\}K_1$

Step 3: Client asks the privilege server for the initial PAC
C → P: $\{C, K_2\}K_p$, {C, MD-5 checksum, Timestamp}K_2
C ← P: $\{K_3,\{PAC, K_3\}K_A\}K_2$

Step 4: Client asks the ticket-granting server for a PAC usable by S
C → T: $\{K_3,\{PAC, K_3\}K_A\}K_3$, {C, MD-5 checksum, Timestamp}K_3
C ← T: $\{K_4,\{PAC, K_4\}K_S\}K_3$

Step 5: Client establishes a key with the application server
C → S: $\{PAC, K_4\}K_S$,{C, MD-5checksum, Timestamp}K_4
C ← S: $\{K_5,$ Timestamp$\}K_4$

Fig. 10-27. From login to authenticated RPC in five easy steps.

When a user sits down at a DCE terminal, the login program asks for his login name. The program then sends this login name to the authentication server in plaintext. The authentication server looks it up in the registry and finds the user's secret key. It then generates a random number for use as a session key and sends back a message encrypted by the user's secret key, K_C, containing the first session key, K_1, and a ticket good for later use with the ticket-granting server. These messages are shown in Fig. 10-27 in step 1. Note that this figure shows 10 messages, and for each the source and destination are given before the message, with the arrow pointing from the source to the destination.

When the encrypted message arrives at the login program, the user is prompted for his password. The password is immediately run through the one-way function that generates secret keys from passwords. As soon as the secret key, K_C has been generated from the password, the password is removed from memory. This procedure minimizes the chance of password disclosure in the event of a client crash. Then the message from the authentication server is decrypted to get the session key and the ticket. When that has been done, the client's secret key can also be erased from memory.

Note that if an intruder intercepts the reply message, it will be unable to decrypt it and thus unable to obtain the session key and ticket inside it. If it spends enough time, the intruder might eventually be able to break the message, but even then the damage will be limited because session keys and tickets are valid only for relatively short periods of time.

In step 2 of Fig. 10-27, the client sends the ticket to the ticket-granting server (in fact, the authentication server under a different name) and asks for a ticket to talk to the privilege server. Except for the initial authentication in step 1, talking to a server in an authenticated way always requires a ticket encrypted with that server's private key. These tickets can be obtained from the ticket-granting server as in step 2.

When the ticket-granting server gets the message, it uses its own private key, K_A to decrypt the message. When it finds the session key, K_1, it looks in the registry and verifies that it recently assigned this key to client C. Since only C knows K_C, the ticket-granting server knows that only C was able to decrypt the reply sent in step 1, and this request must have come from C.

The request also contains an authenticator, basically, a timestamped cryptographically strong checksum of the rest of the message, including the cell name, request, and other fields not shown in the figure. This scheme makes it impossible for an intruder to modify the message without detection, yet does not require the client to encrypt the entire message, which would be expensive for a long message. (In this case the message is not so long, but authenticators are used all the time, for simplicity.) The timestamp in the authenticator guards against an intruder capturing the message and playing in back later because the authentication server will process a request only if it is accompanied by a fresh authenticator.

In this example, we generate and use a new session key at each step. This much paranoia is not required, but the protocol allows it and doing so allows very short lifetimes for each key if the clocks are well synchronized.

Armed with a ticket for the privilege server, the client now asks for a PAC. Unlike a ticket, which contains only the user's login name (in ASCII), a PAC contains the user's identity (in binary as a UUID) plus a list of all the groups to which he belongs. These are important because resources (e.g., a certain printer) are often available to all the members of certain groups (e.g., the marketing department). A PAC is proof that the user named in it belongs to the groups listed in it.

The PAC obtained in step 3 is encrypted with the authentication server/ticket-granting server's secret key. The importance of this choice is that throughout the session, the client may need PACs for several application servers. To get a PAC for use with an application server, the client executes step 4. Here the PAC is sent to the ticket-granting server, which first decrypts it. Since the decryption works, the ticket-granting server is immediately convinced

that the PAC is legitimate, and then it reencrypts it for the client's choice of server, in this case, *S*.

Note that the ticket-granting server does not have to understand the format of a PAC. All it is doing in step 4 is decrypting something encrypted with its own key and then reencrypting it with another key it gets from the registry. While it is at it, the ticket-granting server throws in a new session key, but this action is optional. If the client needs PACs for other servers later, it repeats step 4 with the original PAC as often as needed, specifying the desired server each time.

In step 5, the client sends the new PAC to the application server, which decrypts it, exposing key K_4 used to encrypt the authenticator as well as the client's ID and groups. The server responds with the last key, known only to the client and server. Using this key, the client and server can now communicate securely.

This protocol is complicated, but the complexity is due to the fact that it has been designed to be resistant to a large variety of possible attacks (Bellovin and Merritt, 1991). It also has many features and options that we have not discussed. For example, it can be used to establish secure communication with a server in a distant cell, possibly transiting several potentially untrustworthy cells on every RPC, and can verify the server to the client to prevent **spoofing** (an intruder masquerading as a trusted server).

Once authenticated RPC has been established, it is up to the client and server to determine how much protection is desired. In some cases client authentication or mutual authentication is enough. In others, every packet is authenticated against tampering. Finally, when industrial-strength security is required, all traffic in both directions can be encrypted.

10.6.5. ACLs

Every resource in DCE can be protected by an ACL, modeled after the one in the POSIX 1003.6 standard. The ACL tells who may access the resource and how. ACLs are managed by **ACL managers**, which are library procedures incorporated into every server. When a request comes in to the server that controls the ACL, it decrypts the client's PAC to see what his ID and groups are, and based on these, the ACL, and the operation desired, the ACL manager is called to make a decision about whether to grant or deny access. Up to 32 operations per resource are supported.

Most servers use a standard, but less mathematically inclined, ACL manager, however. This one divides the resources into two categories: simple resources, such as files and data base entries, and containers, such as directories and data base tables, that hold simple resources. It distinguishes between users who live in the cell and foreign users, and for each category further subdivides

them as owner, group, and other. Thus it is possible to specify that the owner can do anything, the local members of the owner's group can do almost anything, unknown users from other cells can do nothing, and so on.

Seven standard rights are supported: read, write, execute, change-ACL, container-insert, container-delete, and test. The first three are as in UNIX. The change-ACL right grants permission to modify the ACL itself. The two container rights are useful to control who may add or delete files from a directory, for example. Finally, test allows a given value to be compared to the resource without disclosing the resource itself. For example, a password file entry might allow users to ask if a given password matched, but would not expose the password file entry itself. An example ACL is shown in Fig. 10-28.

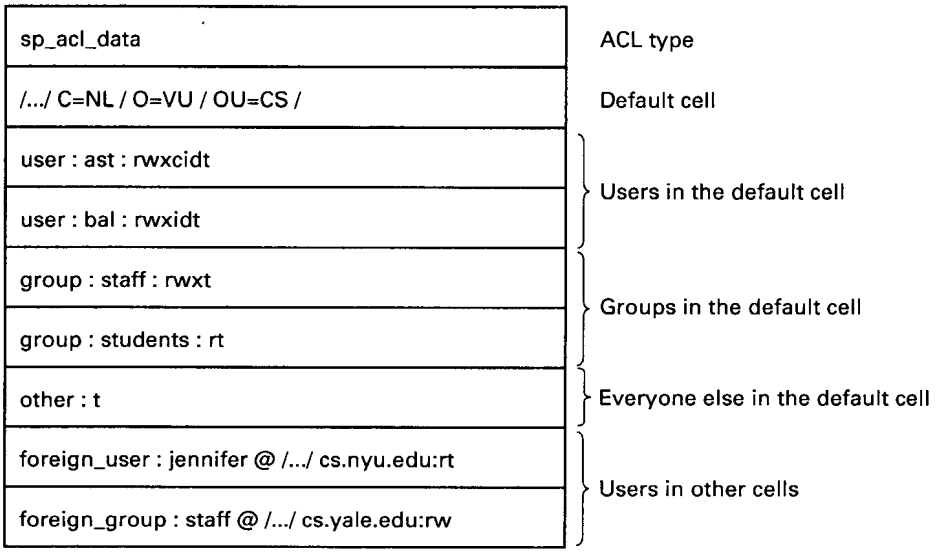

Fig. 10-28. A sample ACL.

In this example, as in all ACLs, the ACL type is specified. This type effectively partitions ACLs into classes based on the type. The default cell is specified next. After that come permissions for two specific users in the default cell, two specific groups in the default cell, and a specification of what all the other users in that cell may do. Finally, the rights of users and groups in other cells can be specified. If a user who does not fit any of the listed categories attempts an access, it will be denied.

To add new users, delete users, or add, delete, or change permissions, an ACL editor program is supplied. To use this program on an access control list, the user must have permission to change the access control list, indicated by the code *c* in the example of Fig. 10-28.

10.7. DISTRIBUTED FILE SYSTEM

The last component of DCE that we will study is **DFS (Distributed File System)** (Kazar et al., 1990). It is a worldwide distributed file system that allows processes anywhere within a DCE system to access all files they are authorized to use, even if the processes and files are in distant cells.

DFS has two main parts: the local part and the wide-area part. The local part is a single-node file system called **Episode**, which is analogous to a standard UNIX file system on a stand-alone computer. One DCE configuration would have each machine running Episode instead of (or in addition to) the normal UNIX file system.

The wide-area part of DFS is the glue that puts all these individual file systems together to form a seamless wide-area file system spanning many cells. It is derived from the CMU AFS system, but has evolved considerably since then. For more information about AFS, see (Howard et al. 1988; Morris et al., 1986, and Satyanarayanan et al., 1985). For Episode itself, see (Chutani et al., 1992).

DFS is a DCE application, and as such can use all the other facilities of DCE. In particular, it uses threads to allow multiple file access requests to be served simultaneously, RPC for communication between clients and servers, DTS to synchronize server clocks, the directory system to allow file servers to be located, and the authentication and privilege servers to protect files.

From DFS' viewpoint, every DCE node is either a file client, a file server, or both. A file client is a machine that uses other machines' file systems. A file client has a cache for storing pieces of recently used files, to improve performance. A file server is a machine with a disk that offers file service to processes on that machine and possibly to processes on other machines as well.

DFS has a number of features that are worth mentioning. To start with, it has uniform naming, integrated with CDS, so file names are location independent. Administrators can move files from one file server to another one within the same cell without requiring any changes to user programs. Files can also be replicated to spread the load more evenly and maintain availability in the event of file server crashes. There is also a facility to automatically distribute new versions of binary programs and other heavily used read-only files to servers (including user workstations).

Episode is a rewrite of the UNIX file system and can replace it on any DCE machine with a disk. It supports the proper UNIX single-system file semantics in the sense that when a write to a file is done and immediately thereafter a read is done, unlike in NFS, the read will see the value just written. It is conformant to the POSIX 1003.1 system-call standard, and also supports POSIX-conformant access control using ACLs, which give flexible protection in large systems. It has also been designed to support fast recovery after a crash by eliminating the need to run the UNIX *fsck* program to repair the file system.

Since many sites may not want to tinker with their existing file system just to run DCE, DFS has been designed to provide seamless integration over machines running 4.3 BSD, System V, NFS, Episode, and other file systems. However, some features of DFS, such as protection by ACLs, will not be available on those parts of the file system supported by non-Episode servers.

10.7.1. DFS Interface

The basic interface to DFS is (intentionally) very similar to UNIX. Files can be opened, read, and written in the usual way, and most existing software can simply be recompiled with the DFS libraries and will work immediately. Mounting of remote file systems is also possible.

The / directory is still the local root, and directories such as */bin*, */lib*, and */usr* still refer to local binary, library, and user directories, as they did in the absence of DFS. A new entry in the root directory is /..., which is the global root. Every file in a DFS system (potentially worldwide), has a unique path from the global root consisting of its cell name concatenated with its name within that cell. In Fig. 10-29(a) we see how a file, *january*, would be addressed globally using an Internet cell name. In Fig. 10-29(b) we see the name of the same file using an X.500 cell name. These names are valid everywhere in the system, no matter what cell the process using the file is in.

(a) Global file name (Internet format)
/.../cs.ucla.edu/fs/usr/ann/exams/january

(b) Global file name (X.500 format)
/.../C=US/O=UCLA/OU=CS/fs/usr/ann/exams/january

(c) Global file name (Cell relative)
/.:/fs/usr/ann/exams/january

(d) Global file name (File system relative)
/:/usr/ann/exams/january

Fig. 10-29. Four ways to refer to the same file.

Using global names everywhere is rather longwinded, so some shortcuts are available. A name starting with */.:/fs* means a name in the current cell starting from the *fs* junction (the place where the local file system is mounted on the global DFS tree), as shown in Fig. 10-29(c). This usage can be shortened even further as given in Fig. 10-29(d)

Unlike in UNIX, protection in DFS uses ACLs instead of the three groups of RWX bits, at least for those files managed by Episode. Each file has an ACL telling who can access it and how. In addition, each directory has three ACLs These ACLs give access permissions for the directory itself, the files in the directory, and the directories in the directory, respectively.

ACLs in DFS are managed by DFS itself because the directories are DFS

objects. An ACL for a file or directory consists of a list of entries. Each entry describes either the owner, the owner's group, other local users, foreign (i.e., out-of-cell) users or groups, or some other category, such as unauthenticated users. For each entry, the allowed operations are specified from the set: read, write, execute, insert, delete, and control. The first three are the same as in UNIX. Insert and delete make sense on directories, and control makes sense for I/O devices subject to the IOCTL system call.

DFS supports four levels of aggregation. At the bottom level are individual files. These can be collected into directories in the usual way. Groups of directories can be put together into **filesets**. Finally, a collection of filesets forms a disk partition (an **aggregate** in DCE jargon).

A fileset is normally a subtree in the file system. For example, it may be all the files owned by one user, or all the files owned by the people in a certain department or project. A fileset is a generalization of the concept of the file system in UNIX (i.e., the unit created by the *mkfs* program). In UNIX, each disk partition holds exactly one file system, whereas in DFS it may hold many filesets.

The value of the fileset concept can be seen in Fig. 10-30. In Fig. 10-30(a), we see two disks (or two disk partitions) each with three empty directories. In the course of time, files are created in these directories. As it turns out, disk 1 fills up much faster than disk 2, as shown in Fig. 10-30(b). If disk 1 becomes full while disk 2 still has plenty of space, we have a problem.

The DFS solution is to make each of the directories *A*, *B*, and *C* (and their subdirectories) a separate fileset. DFS allows filesets to be moved, so the system administrator can rebalance the disk space simply by moving directory *A* to disk 2, as shown in Fig. 10-30(c). As long as both disks are in the same cell, no global names change, so everything continues to work after the move as it did before.

In addition to moving filesets, it is also possible to replicate them. One replica is designated as the master, and is read/write. The others are slaves and are read only. Filesets, rather than disk partitions, are the units that are manipulated for motion, replication, backup, and so on. For UNIX systems, each disk partition is considered to be an aggregate with one fileset. Various commands are available to system administrators for managing filesets.

10.7.2. DFS Components in the Server Kernel

DFS consists of additions to both the client and server kernels, as well as various user processes. In this section and the following ones, we will describe this software and what it does. An overview of the parts is presented in Fig. 10-31.

On the server we have shown two file systems, the native UNIX file system and, alongside it, the DFS local file system, Episode. On top of both of them is

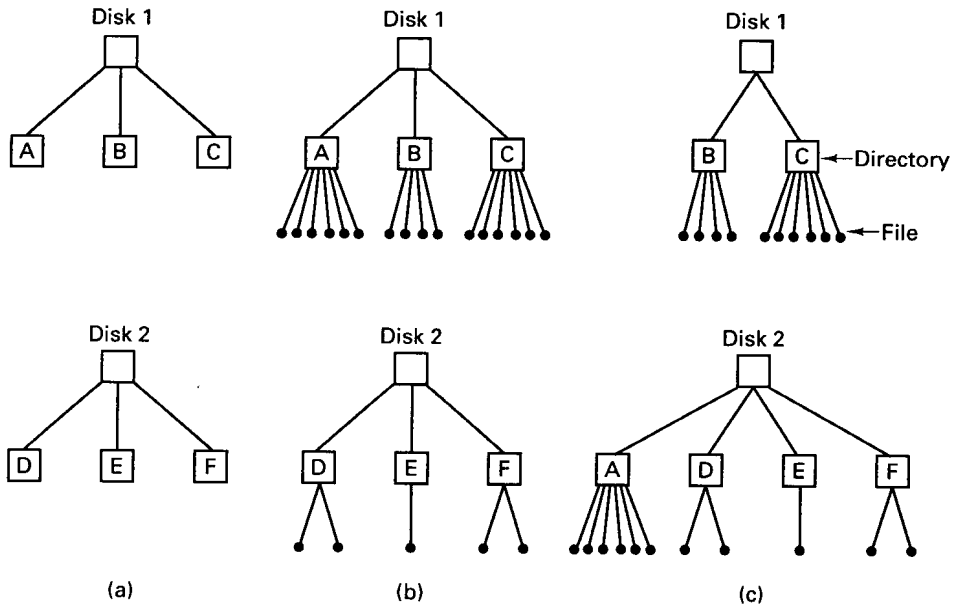

Fig. 10-30. (a) Two empty disks. (b) Disk 1 fills up faster than disk 2. (c) Configuration after moving one fileset.

the token manager, which handles consistency. Further up are the file exporter, which manages interaction with the outside world, and the system call interface, which manages interaction with local processes. On the client side, the major new addition is the cache manager, which caches file fragments to improve performance.

Let us now examine Episode. As mentioned above, it is not necessary to run Episode, but it offers some advantages over conventional file systems. These include ACL-base protection, fileset replication, fast recovery, and files of up to 2^{42} bytes. When the UNIX file system is used, the software marked "Extensions" in Fig. 10-31(b) handles matching the UNIX file system interface to Episode, for example, converting PACs and ACLs into the UNIX protection model.

An interesting feature of Episode is its ability to clone a fileset. When this is done, a virtual copy of the fileset is made in another partition and the original is marked "read only." For example, it might be cell policy to make a read-only snapshot of the entire file system every day at 4 A.M. so that even if someone deleted a file inadvertently, he could always go back to yesterday's version.

Episode does cloning by copying all the file system data structures (i-nodes in UNIX terms) to the new partition, simultaneously marking the old ones as read only. Both sets of data structures point to the same data blocks, which are not

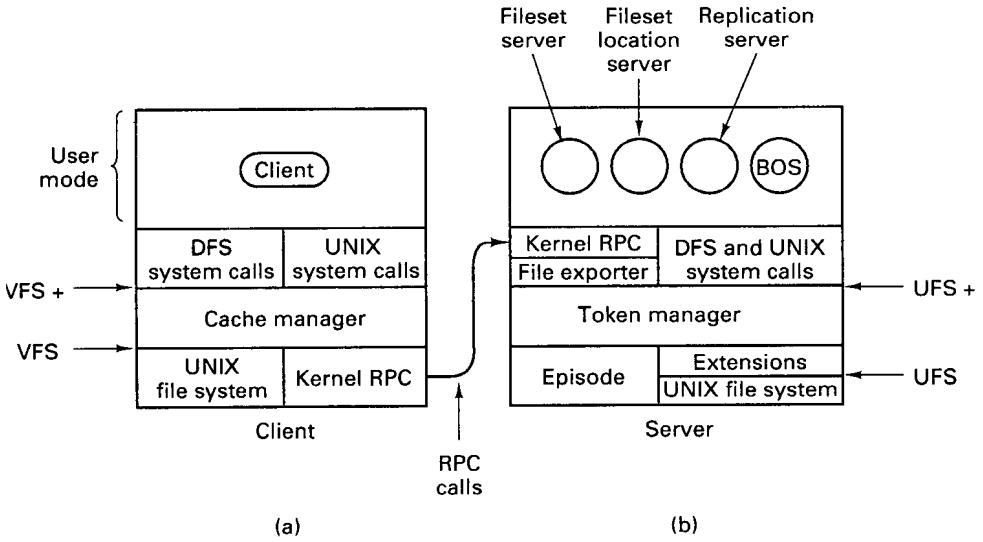

Fig. 10-31. Parts of DFS. (a) File client machine. (b) File server machine.

copied. As a result, cloning can be done extremely quickly. An attempt to write on the original file system is refused with an error message. An attempt to write on the new file system succeeds, with copies made of the new blocks.

Episode was designed for highly concurrent access. It avoids having threads take out long-term locks on critical data structures to minimize conflicts between threads needing access to the same tables. It also has been designed to work with asynchronous I/O, providing an event notification system when I/O completes.

Traditional UNIX systems allow files to be any size, but limit most internal data structures to fixed-size tables. Episode, in contrast, uses a general storage abstraction called an **a-node** internally. There are a-nodes for files, filesets, ACLs, bit maps, logs, and other items. Above the a-node layer, Episode does not have to worry about physical storage (e.g., a very long ACL is no more of a problem than a very long file). An a-node is a 252-byte data structure, this number being chosen so that four a-nodes and 16 bytes of administrative data fit in a 1K disk block.

When an a-node is used for a small amount of data (up to 204 bytes) the data are stored directly in the a-node itself. Small objects, such as symbolic links and many ACLs often fit. When an a-node is used for a larger data structure, such as a file, the a-node holds the addresses of eight blocks full of data and four indirect blocks that point to disk blocks containing yet more addresses.

Another noteworthy aspect of Episode is how it deals with crash recovery.

Traditional UNIX systems tend to write changes to bit maps, i-nodes, and directories back to the disk quickly to avoid leaving the file system in an inconsistent state in the event of a crash. Episode, in contrast, writes a log of these changes to disk instead. Each partition has its own log. Each log entry contains the old value and the new value. In the event of a crash, the log is read to see which changes have been made and which have not been. The ones that have not been made (i.e., were lost on account of the crash) are then made. It is possible that some recent changes to the file system are still lost (if their log entries were not written to disk before the crash), but the file system will always be correct after recovery.

The primary advantage of this scheme is that using it the recovery time is proportional to the length of the log rather than proportional to the size of the disk, as it is when the UNIX *fsck* program is run to repair a potentially sick disk in traditional systems.

Getting back to Fig. 10-31, the layer on top of the file systems is the token manager. Since the use of tokens is intimately tied to caching, we will discuss tokens when we come to caching in the next section. At the top of the token layer, an interface is supported that is an extension of the Sun NFS VFS interface. VFS supports file system operations, such as mounting and unmounting, as well as per file operations such as reading, writing, and renaming files. These and other operations are supported in VFS+. The main difference between VFS and VFS+ is the token management.

Above the token manager is the file exporter. It consists of several threads whose job it is to accept and process incoming RPCs that want file access. The file exporter handles requests not only for Episode files, but also for all the other file systems present in the kernel. It maintains tables keeping track of the various file systems and disk partitions available. It also handles client authentication, PAC collection, and establishment of secure channels. In effect, it is the application server described in step 5 of Fig. 10-27.

10.7.3. DFS Components in the Client Kernel

The main addition to the kernel of each client machine in DCE is the DFS cache manager. The goal of the cache manager is to improve file system performance by caching parts of files in memory or on disk, while at the same time maintaining true UNIX single-system file semantics. To make it clear what the nature of the problem is, let us briefly look at UNIX semantics and why this is an issue.

In UNIX (and all uniprocessor operating systems, for that matter), when one process writes on a file, and then signals a second process to read the file, the value read *must* be the value just written. Getting any other value violates the semantics of the file system.

This semantic model is achieved by having a single buffer cache inside the operating system. When the first process writes on the file, the modified block goes into the cache. If the cache fills up, the block may be written back to disk. When the second process tries to read the modified block, the operating system first searches the cache. Failing to find it there, it tries the disk. Because there is only one cache, under all conditions, the correct block is returned.

Now consider how caching works in NFS. Several machines may have the same file open at the same time. Suppose that process 1 reads part of a file and caches it. Later, process 2 writes that part of the file. The write does not affect the cache on the machine where process 1 is running. If process 1 now rereads that part of the file, it will get an obsolete value, thus violating the UNIX semantics. This is the problem thar DFS was designed to solve.

Actually, the problem is even worse than just described, because directories can also be cached. It is possible for one process to read a directory and delete a file from it. Nevertheless, another process on a different machine may subsequently read its previously cached copy of the directory and still see the now-deleted file. While NFS tries to minimize this problem by rechecking for validity frequently, errors can still occur.

DFS solves this problem using tokens. To perform any file operation, a client makes a request to the cache manager, which first checks to see if it has the necessary token and data. If it has both the token and the data, the operation may proceed immediately, without contacting the file server. If the token is not present, the cache manager does an RPC with the file server asking for it (and for the data). Once it has acquired the token, it may perform the operation.

Tokens exist for opening files, reading and writing files, locking files, and reading and writing file status information (e.g., the owner). Files can be opened for reading, writing, both, executing, or exclusive access. Open and status tokens apply to the entire file. Read, write, and lock tokens, in contrast, refer only to some specific byte range. Tokens are granted by the token manager in Fig. 10-31(b).

Figure 10-32 gives an example of token usage. In message 1, client 1 asks for a token to open some file for reading, and also asks for a token (and the data) for the first chunk of the file. In message 2, the server grants both tokens and provides the data. At this point, client 1 may cache the data it has just received and read it as often as it likes. The normal transfer unit in DFS is 64K bytes.

A little later, client 2 asks for a token to open the same file for reading and writing, and also asks for the initial 64K of data to selectively overwrite part of it. The server cannot just grant these tokens, since it no longer has them. It must send message 4 to client 1 asking for them back.

As soon as is reasonably convenient, client 1 must return the revoked tokens to the server. After it gets the tokens back, the server gives them to client 2, which can now read and write the data to its hearts' content without notifying

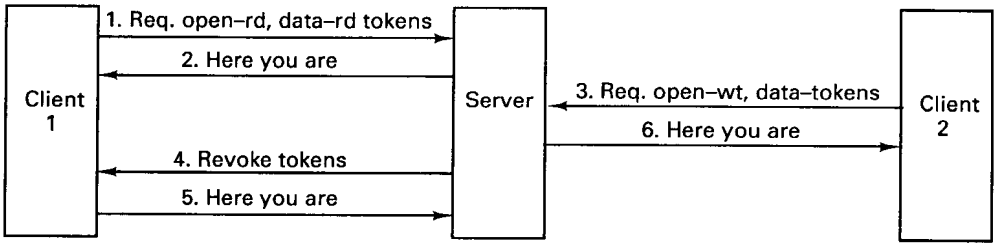

Fig. 10-32. An example of token revocation.

the server. If client 1 asks for the tokens back, the server will instruction client 2 to return the tokens along with the updated data, so these data can be sent to client 1. In this way, single-system file semantics are maintained.

To maximize performance, the server understands about token compatibility. It will not simultaneously issue two write tokens for the same data, but it will issue two read tokens for the same data, provided that both clients have the file open only for reading.

Tokens are not valid forever. Each token has an expiration time, usually two minutes. If a machine crashes and is unable (or unwilling) to return its tokens when asks, the file server can just wait two minutes and then act like they have been returned.

On the whole, caching is transparent to user processes. However, calls are available for certain cache management functions, such as prefetching a file before it is used, flushing the cache, disk quota management, and so on. The average user does not need these, however.

Two differences between DFS and its predecessor, AFS, are worth mentioning. In AFS, entire files were transferred, instead of 64K chunks. This strategy made a local disk essential, since files were often too large to cache in memory. With a 64K transfer unit, local disks are no longer required.

A second difference is that in AFS part of the file system code was in the kernel and part was in user space. Unfortunately, the performance left much to be desired, so in DFS it is all in the kernel.

10.7.4. DFS Components in User Space

We have now finished discussing those parts of DFS that run in the server and client kernels. Let us now briefly discuss those parts of it that run in user space (see Fig. 10-31). The **fileset server** manages entire filesets. Each fileset contains one or more directories and their files and must be located entirely with one partition. Filesets can be mounted to form a hierarchy. Each fileset has a disk quota to which it is restricted.

The fileset server allows the system administrator to create, delete, move, duplicate, clone, backup, or restore an entire fileset with a single command. Each of these operations locks the fileset, does the work, and releases the lock. A fileset is created to set up a new administrative unit for future use. When it is no longer needed, it can be deleted.

A fileset can be moved from one machine to another to balance the load, both in terms of disk storage and in terms of number of requests per second that must be handled. When a fileset is copied but the original is not deleted, a duplicate is created. Duplicates are supported, and provide both load balancing and fault tolerance. Only one copy is writable.

Cloning, as described above, just copies the status information (a-nodes) but does not copy the data. Duplication makes a new copy of the data as well. A clone must be in the same disk partition as the original. A duplicate can be anywhere (even if a different cell).

Backup and restore are functions that allow a fileset to be linearized and copied to or from tape for archival storage. These tapes can be stored in a different building to make it possible to survive not only disk crashes, but also fires, floods and other disasters.

The fileset server also can provide information about filesets, manipulate fileset quotas, and perform other management functions.

The **fileset location server** manages a cell-wide replicated data base that maps fileset names onto the names of the servers that hold the filesets. If a fileset is replicated, all the servers having a copy can be found.

The fileset location server is used by cache managers to locate filesets. When a user program accesses a file for the first time, its cache manager asks the fileset location server where it can find the fileset. This information is cached for future use.

Each entry in the data base contains the name of the fileset, the type (read/write, read-only, or backup), the number of servers holding it, the addresses of these servers, the fileset's owner and group information, information about clones, cache timeout information, token timeout information, and other administrative information.

The **replication server** keeps replicas of filesets up to date. Each fileset has one master (i.e., read/write) copy and possibly one or more slave (i.e., read-only) copies. The replication server runs periodically, scanning each replica to see which files have been changed since the replica was last updated. These files are replaced from the current files in the master copy. After the replication server has finished, all the replicas are up to date.

The **Basic Overseer Server** runs on every server machine. Its job is to make sure that the other servers are alive and well. If it discovers that some servers have crashed, it brings up new versions. It also provides an interface for system administrators to stop and start servers manually.

10.8. SUMMARY

DCE is a different approach to building a distributed system than that taken by Amoeba, Mach, and Chorus. Instead of starting from scratch with a new operating system running on the bare metal, DCE provides a layer on top of the native operating system that hides the differences among the individual machines, and provides common services and facilities that unify a collection of machines into a single system that is transparent in some (but not all) respects. DCE runs on top of UNIX and other operating systems.

DCE supports two facilities that are used heavily, both within DCE itself and by user programs—threads and RPC. Threads allow multiple control streams to exist within one process. Each has its own program counter, stack, and registers, but all the threads in a process share the same address space, file descriptors, and other process resources.

RPC is the basic communication mechanism used throughout DCE. It allows a client process to call a procedure on a remote machine. DCE provides a variety of options for a client to select and bind to a server.

DCE supports four major services (and several minor ones) that can be accessed by clients. These are the time, directory, security, and file services. The time service attempts to keep all the clocks with a DCE system synchronized within known limits. An interesting feature of the time service is that it represents times not as single values, but as intervals. As a result, it is possible that when comparing two times it is not possible to say unambiguously which came first.

The directory service stores the names and locations of all kinds of resources and allows clients to look them up. The CDS holds local names (within the cell). The GDS holds global (out-of-cell) names. Both the DNS and X.500 naming systems are supported. Names form a hierarchy. The directory service is, in fact, a replicated, distributed data base system.

The security service allows clients and servers to authenticate each other and perform authenticated RPC. The heart of the security system is a way for clients to be authenticated and receive PACs without having their passwords appear on the network, not even in encrypted form. PACs allow clients to prove who they are in a convenient and foolproof way.

Finally, the distributed file system provides a single, system-wide name space for all files. A global file name consists of a cell name followed by a local name. The DCE file system consists (optionally) of the DCE local file system, Episode, plus the file exporter, which makes all the local file systems visible throughout the system. Files are cached using a token scheme that maintains the traditional single-system file semantics.

Although DCE provides many facilities and tools, it is not complete and probably never will be. Some areas in which more work is needed are

specification and design techniques and tools, debugging aids, runtime management, object orientation, atomic transactions, and multimedia support.

PROBLEMS

1. A university is installing DCE on all computers on campus. Suggest at least two ways of dividing the machines into cells.
2. DCE threads can be in one of four states, as described in the text. In two if these states, ready and waiting, the thread is not running. What is the difference between these states?
3. In DCE, some data structures are process wide, and others are per thread. Do you think that the UNIX environment variables should be process wide, or should every thread have its own environment? Defend your viewpoint.
4. A condition variable in DCE is always associated with a mutex. Why?
5. A programmer has just written a piece of multithreaded code that uses a private data structure. The data structure may not be accessed by more than one thread at a time. Which kind of mutex should be used to protect it?
6. Name two plausible attributes that the template for a thread might contain.
7. Each IDL file contains a unique number (e.g., produced by the *uuidgen* program). What is the value of having this number?
8. Why does IDL require the programmer to specify which parameters are input and which are output?
9. Why are RPC endpoints assigned dynamically by the RPC daemon instead of statically? A static assignment would surely be simpler.
10. A DCE programmer needs to time a benchmark program. It calls the local clock time procedure (on its own machine) and learns that the time is 15:30:00.0000. Then it starts the benchmark immediately. When the benchmark finishes, it calls the time procedure again and learns that the time is now 15:35:00.0000. Since it used the same clock, on the same machine, for both measurements, is it safe to conclude that the benchmark ran for 5 minutes ($\pm$ 0.1 msec)? Defend your answer.
11. In a DCE system, the uncertainty in the clock is $\pm$ 5 sec. If source file has time interval 14:20:30 to 14:20:40 and its binary file has time 14:20:32 to 14:20:42, what should *make* do if called at 14:20:38? Suppose that *make* takes 1 sec to do its job. What happens if *make* is called a second time a few minutes later?

12. A time clerk asks four time servers for the current time. The responses are

16:00:10.075 ± 0.003,
16:00:10.085 ± 0.003,
16:00:10.069 ± 0.007,
16:00:10.070 ± 0.010,

What time does the clerk set its clock to?

13. Can DTS run without a UTC source connected to any of the machines in the system? What are the consequences of doing this?

14. Why do distinguished names have to be unique, but not RDNs?

15. Give the X.500 name for the sales department at Kodak in Rochester, NY.

16. What is the function of the CDS clerk? Would it have been possible to have designed DCE without the CDS clerks? What would the consequences have been?

17. CDS is a replicated data base system. What happens if two processes try simultaneously to change the same item in different replicas to different values?

18. If DCE did not support the Internet naming system, which parts of Fig. 10-24 could be dispensed with?

19. During the authentication sequence, a client first acquires a ticket and later a PAC. Since both of them contain the user's ID, why go to the trouble of acquiring a PAC once the necessary ticket has been obtained?

20. What is the difference, if any, between the following messages:

$\{\{\text{message}\}K_A\}K_C$ and
$\{\{\text{message}\}K_C\}K_A$.

21. The authentication protocol described in the text allows an intruder to send arbitrarily many messages to the authentication server in an attempt to get a reply containing an initial session key. What weakness does this observation introduce into the system? (*Hint*: You may assume that all messages contain a timestamp and other information, so it is easy to tell a valid message from random junk.)

22. The text discussed only the case of security within a single cell. Propose a way to do authentication for RPC when the client is in the local cell and server is in a remote cell.

23. Tokens can expire in DFS. Does this require synchronized clocks using DFS?

24. An advantage of DFS over NFS is that DFS preserves single-system file semantics. Name a disadvantage.

25. Why must a clone of a fileset be in the same disk partition?

Index

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

X

Y